PRIVACY-ENHANCED SYNTHETIC DATA GENERATION FOR AUTONOMOUS VEHICLE TRAINING

INDIVIDUAL THESIS

Wijesinghe R.D.P.T.

(IT22066916)

Individual component: Risk and Compliance

Group ID: R26-CS-010

Report submitted in partial fulfilment of the requirements for the

Bachelor of Science (Honours) Degree in Information Technology

Specialising in Cyber Security

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools as the module policy requires. I remain responsible for the accuracy, originality, citations, code, data, analyses and conclusions in this thesis.

Student name: Wijesinghe R.D.P.T.

Student ID: IT22066916

Signature: _____ Date: 26/10/2026

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Prof. Anuradha Jayakody		26/10/26
Co-supervisor: Mr. Dinith Primal		26/10/2026

ABSTRACT

A privacy-enhanced pipeline for synthetic driving data makes many claims. It claims a privacy budget, intact data and records that cannot be changed. These claims are usually checked only by the team that made them. This thesis presents the Risk and Compliance component of a four-part framework for synthetic autonomous vehicle data. The component does not add a privacy mechanism of its own. It checks the mechanisms of the other components with code written separately from them. It then maps the evidence to the General Data Protection Regulation (GDPR) and issues a signed compliance certificate. The component runs seven checks. It verifies HMAC-SHA256 signatures and audits each reported ε against per-step budget logs. It reconciles the calibration noise log, runs a reconstruction attack and checks budget composition. It also re-hashes the dataset, measures k-anonymity and scans released files for leakage. All twelve signed audit reports were valid, and a changed copy was rejected. Every reported training ε matched the last of 3,090 logged steps. The composition check found that an earlier calibration release spent $\varepsilon = 12.6$ while it reported 1.8. After the fix, the seven queries composed to exactly 1.8. On the corrected release, the observed reconstruction error was 0.94 of the error that a correct Laplace mechanism gives. All 40,000 delivered files matched their manifests. The metadata reached $k = 5$ with no unique records, and the leakage scan found nothing. The final report verified eight GDPR controls with an overall PASS. It was fingerprinted with SHA-256, signed and issued as a certificate with a QR code. A later code review found a fixed public seed in the calibration noise, which no check had detected. It was corrected before the dataset was captured again. The results show that independent checks find faults that the producing team misses.

Keywords: compliance verification, differential privacy, GDPR, audit integrity, synthetic data, autonomous vehicles

ACKNOWLEDGEMENTS

I would like to thank my supervisor, **Prof. Anuradha Jayakody**, and my co-supervisor, **Mr. Dinith Primal**, for their guidance and feedback throughout this research. Their advice helped me define what an independent check can and cannot prove.

I also thank my team members, Priyadarshi S.A.S.D. (IT22309556), Yubitha S. (IT22541284) and Apeksha M.K.S.R. (IT22110220), for their support and cooperation. They shared their outputs, answered many questions about their file formats and acted on the findings of my checks. I thank the members of the progress review panels for their comments. Finally, I thank the developers of Python, pandas, ReportLab and CARLA. Their open-source tools made this work possible.

TABLE OF CONTENTS

1 INTRODUCTION 1
1.1 Overall project context 1
1.2 Individual component context 1
1.3 Problem statement 2
1.4 Research gap 3
1.5 Aim and objectives 3
1.6 Research questions or hypotheses 4
1.7 Scope limitations and assumptions 4
1.8 Individual contribution 5
1.9 Thesis structure 5
2 LITERATURE REVIEW AND RELATED WORK 6
2.1 Review method and scope 6
2.2 Domain and theoretical foundations 6
2.3 Related research 7
2.4 Existing systems products and techniques 8
2.5 Gap synthesis and conceptual framework 9
3 RESEARCH METHODOLOGY 11
3.1 Research design 11
3.2 Data participants or study materials 12
3.3 Experimental or study procedure 12
3.4 Measures and evaluation metrics 13
3.5 Data analysis 14
3.6 Ethics privacy security and responsible AI 15
3.7 Reproducibility and research artefacts 15
4 PRODUCT AND SOLUTION DESIGN 17
4.1 Requirements 17
4.2 Component architecture within the complete solution 19
4.3 Design alternatives and decisions 20
4.4 Detailed design 21
4.5 Standards and professional practice 24

5 IMPLEMENTATION INTEGRATION AND TESTING 25
5.1 Development environment and technology stack 25
5.2 Component implementation 25
5.2.1 Implementation challenges 27
5.3 Integration with the other components 28
5.4 Verification and software testing 29
5.5 Deployment and operation 30
6 RESULTS AND VALIDATION 32
6.1 Research results 32
6.1.1 Integrity of the signed evidence 32
6.1.2 Privacy budget audit 32
6.1.3 Calibration channel 34
6.1.4 Dataset integrity and metadata 36
6.1.5 Poisoning verdict 37
6.2 Product and technical results 37
6.3 User or stakeholder validation 39
6.4 Integration results 40
6.5 Summary of findings 41
7 DISCUSSION 42
7.1 Interpretation of findings 42
7.2 Comparison with literature baselines and existing solutions 43
7.3 Research contribution 43
7.4 Product contribution 44
7.5 Limitations and threats to validity 44
7.6 Implications 45
8 COMMERCIALISATION AND SUSTAINABILITY 46
8.1 Value proposition and target users 46
8.2 Market and competitive position 46
8.3 Business or sustainability model 47
8.4 Scalability intellectual property and compliance 47
8.5 Contribution to the Sustainable Development Goals 48

9 CONCLUSION AND FUTURE DIRECTIONS 49
9.1 Conclusion 49
9.2 Achievement of objectives 49
9.3 Individual research and product contributions 50
9.4 Future directions 50
REFERENCES 52
GLOSSARY 55
APPENDICES 56
Appendix A Extended results 56
Appendix B Academic integrity and AI use disclosure 57

LIST OF FIGURES

Figure 1.1: Overall system overview of the R26-CS-010 framework 2

Figure 2.1: Conceptual framework of the Risk and Compliance component 10

Figure 3.1: Research design and audit procedure of the component 11

Figure 4.1: Position of Risk and Compliance in the group pipeline 19

Figure 4.2: Checks of the Risk and Compliance component, their inputs and their results on the final outputs 21

Figure 6.1: Target and final logged ε of the three private configurations..... 33

Figure 6.2: Composed calibration budget of the earlier and corrected releases 35

Figure 6.3: Observed and expected reconstruction error of the two releases 35

Figure 6.4: Frames accepted and excluded by the poisoning gate in each town 37

Figure 6.5: Compliance certificate issued for the final outputs 39

LIST OF TABLES

Table 2.1: Existing systems and techniques compared with the needs of the component 8
Table 2.2: Research gaps addressed by the Risk and Compliance component 9
Table 3.1: Study materials received from the other components 12
Table 3.2: Measures and evaluation metrics 13
Table 4.1: Functional requirements of the Risk and Compliance component 17
Table 4.2: Trace from the SRS requirements to the final design 18
Table 4.3: Design alternatives and decisions 20
Table 4.4: GDPR provisions mapped to controls, owners and evidence checks 23
Table 5.1: Development environment and technology stack 25
Table 5.2: Command-line tools of the component 25
Table 5.3: Checks combined in the compliance report 27
Table 5.4: Files exchanged by the Risk and Compliance component 28
Table 5.5: Unit tests by module 29
Table 5.6: Verification of the functional requirements 30
Table 6.1: Verification of the signed audit reports 32
Table 6.2: Budget audit of each configuration, with all values in ε (seed 42; seeds 43 and 44 identical) 33
Table 6.3: Calibration checks on the earlier and corrected releases 34
Table 6.4: Dataset integrity, anonymity and leakage results 36
Table 6.5: Final compliance report 37
Table 6.6: Summary of findings 41
Table 9.1: Achievement of objectives 49
Table A.1: Manifest verification per town 56
Table A.2: Composition check of the corrected calibration release 56
Table A.3: Poisoning gate result per town 57

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AES	Advanced Encryption Standard
AI	Artificial Intelligence
AUC	Area Under the ROC Curve
AV	Autonomous Vehicle
CARLA	Car Learning to Act (open-source driving simulator)
CBC	Cipher Block Chaining
CI	Continuous Integration
CSV	Comma-Separated Values
DP	Differential Privacy
DP-SGD	Differentially Private Stochastic Gradient Descent
EDPB	European Data Protection Board
FID	Fréchet Inception Distance
FR	Functional Requirement
GDPR	General Data Protection Regulation
GPU	Graphics Processing Unit
HMAC	Hash-based Message Authentication Code
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
KITTI	Karlsruhe Institute of Technology and Toyota Technological Institute (dataset)
mAP	Mean Average Precision
MIA	Membership Inference Attack
NFR	Non-Functional Requirement
NIST	National Institute of Standards and Technology
PDF	Portable Document Format
QR	Quick Response (code)
RBAC	Role-Based Access Control
RDP	Rényi Differential Privacy
ROC	Receiver Operating Characteristic
RQ	Research Question
SDG	Sustainable Development Goal

Abbreviation	Meaning
SHA	Secure Hash Algorithm
SRS	Software Requirements Specification
YOLO	You Only Look Once (object detector)

1 INTRODUCTION

1.1 Overall project context

Autonomous vehicles use machine learning to detect objects and plan their movement. These models need large amounts of labeled sensor data. Public datasets such as KITTI [1] and the Waymo Open Dataset [2] were recorded on public roads. Their cameras capture faces, license plates and repeated routes. Under the General Data Protection Regulation (GDPR), recordings of identifiable people are personal data [3]. A company that trains on such data must protect it and must be able to show that it did.

Simulators such as CARLA [4] reduce the need for real recordings. However, a simulator is still tuned with real statistics, such as traffic density and weather. Models trained on its output can also reveal their training records [5]. Stadler et al. [6] showed that synthetic data does not protect its source without a formal privacy mechanism.

Project R26-CS-010 builds a privacy-enhanced synthetic data generation framework with four components. Privacy Integration (IT22309556) applies differential privacy to the calibration statistics and to model training. Dataset Generation (IT22541284) captures CARLA frames and protects them with SHA-256 hashing, AES-256 encryption and role-based access control. Utility Evaluation (IT22110220) decrypts the data, checks it for poisoning and measures its value for training a detector. Risk and Compliance (IT22066916) is the subject of this thesis. It checks the claims of the other three components and maps the evidence to the GDPR.

1.2 Individual component context

Each of the other components produces evidence about its own work. Privacy Integration signs an audit report for every training run and logs ε at every step. Dataset Generation ships a SHA-256 manifest with each encrypted town. Utility Evaluation writes a poisoning verdict before training. These files make claims, but a claim checked only by its author is weak evidence. A bug in a signer, a logger or a noise function can pass unnoticed by the same team.

The Risk and Compliance component acts as the auditor of the pipeline. It sits at the end of the data flow and reads the evidence of the other three components. It does not

reuse their signing, hashing or noise code. It writes its own checks and runs them on the final outputs. A failed manifest check stops the pipeline before any training starts. At the end, the component writes a compliance report and a signed certificate. Figure 1.1 shows how the four components connect.

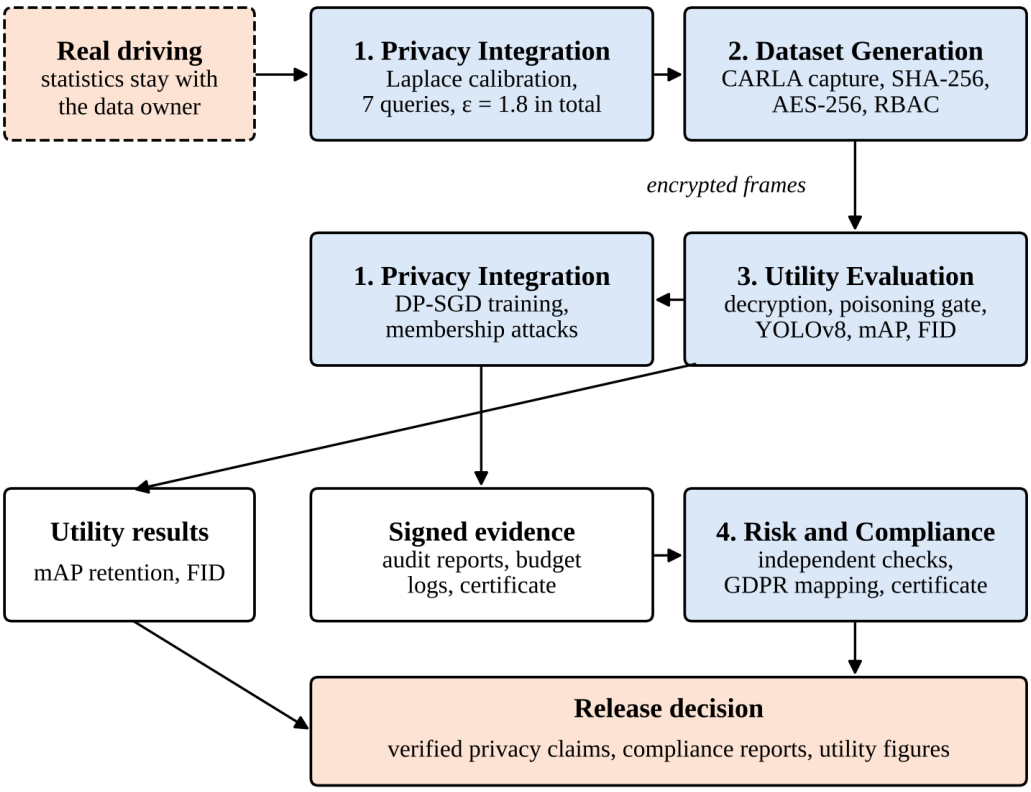


Figure 1.1: Overall system overview of the R26-CS-010 framework

1.3 Problem statement

Privacy-preserving pipelines report numbers such as ε and verdicts such as PASS. The people who read these numbers usually cannot check them. A data protection officer receives a privacy budget but not the evidence behind it. An auditor receives a signed report but not a way to test the signature. Recent audits of differential privacy software found errors that broke the guarantee [7], [8]. In these cases, the reported ε still looked correct. In addition, a list of security controls does not show which GDPR duty each one supports. As a result, a synthetic data pipeline can claim compliance without evidence that a third party can check.

1.4 Research gap

Earlier work leaves four gaps that matter for this component. First, privacy audits of machine learning focus on attacks against trained models [9], [10]. They rarely check the records that report the budget, or how budgets add up across separate queries. Second, simulator calibration is treated as configuration, so its privacy cost is not audited [4]. Third, compliance tools for machine learning measure leakage [11]. They do not link each control to a GDPR article with a verdict. Fourth, compliance outputs are often plain documents that can be edited without trace. Section 2.5 explains these gaps in more detail.

1.5 Aim and objectives

The aim of this research is to verify the privacy and security claims of the pipeline independently. The results should be mapped to GDPR duties in a form that a third party can check. The project proposal set two measurable targets that relate to this component. The success rate of a membership inference attack should stay at or below 55%. A re-identification attack should succeed on no more than 10% of samples. Section 1.7 explains how the scope changed during the project. The specific objectives are:

1. To build a compliance baseline that maps GDPR articles to pipeline controls and to the checks that give evidence.
2. To verify the signed audit reports with independent code and show that a changed report is rejected.
3. To audit every reported privacy budget against its per-step log and the hard limit of $\varepsilon = 10$.
4. To check the calibration channel by reconciling its noise log, measuring reconstruction error and checking budget composition.
5. To verify the integrity of the delivered dataset and to check its metadata for singling out and leakage.
6. To combine the results into a compliance report with honest verdicts, and to issue a signed certificate.
7. To run every check automatically inside the group pipeline.

1.6 Research questions or hypotheses

The main research question of this component is:

RQ: Can an independent component verify the privacy and security claims of a synthetic data pipeline and map them to GDPR duties?

The answer should take a form that others can check.

The question was divided into four sub-questions:

- **RQ1:** Can signed privacy evidence be verified without trusting the code that produced it?
- **RQ2:** Does the calibration channel spend the budget it reports, with noise of the size that the Laplace mechanism requires?
- **RQ3:** Do the delivered dataset and its metadata keep their integrity and avoid singling out and leakage?
- **RQ4:** Can the results form a GDPR mapping and a certificate that show any change without overstating compliance?

The working hypothesis was that separately written checks would find at least one fault that the producer had missed.

1.7 Scope limitations and assumptions

The Software Requirements Specification (SRS) gave this component two attacks. These were a shadow-model membership inference attack and a re-identification attack on synthetic trajectories. During implementation, Privacy Integration built and ran the membership inference study. That component owns the trained models and their training code. As a result, the scope of Risk and Compliance moved to independent verification of the evidence. An independent repeat of the membership inference study is planned. It was not run, so no result from it is claimed. The re-identification attack was not implemented. The final pipeline holds no individual real records to match against, because the calibration source contains only aggregates.

The work makes three assumptions. First, the HMAC key was shared with this component through a separate channel and was kept secret. Second, the checks cover

the files listed in Section 3.2 and not the whole software of the other components. Third, the GDPR mapping is a technical record. It does not replace the legal judgment of a data controller.

1.8 Individual contribution

The author designed and built the whole Risk and Compliance component. The main contributions are:

- A GDPR compliance baseline with eight mappings, each linked to an owner and to a check that gives evidence.
- Independent verification of HMAC-SHA256 signatures and SHA-256 digests, with a tamper test.
- A budget audit that compares each reported ε with its per-step log and checks monotonicity and the limit.
- A calibration check with three parts: noise log reconciliation, a reconstruction attack and a composition check.
- A manifest check that re-hashes every decrypted file without needing the AES key.
- A k-anonymity measure for the frame metadata and a regular-expression leakage scan of released files.
- A compliance report with PASS, FAIL and INCOMPLETE verdicts, and a signed PDF certificate with a QR code.
- Seven pipeline stages, including a manifest gate, and 79 unit tests.

1.9 Thesis structure

Chapter 2 reviews related work and builds the conceptual framework. Chapter 3 explains the research design, the study material, the procedure and the measures. Chapter 4 presents the requirements and the design of the component. Chapter 5 covers implementation, integration and testing. Chapter 6 reports the results, and Chapter 7 discusses them. Chapter 8 considers commercialization and sustainability. Chapter 9 concludes the thesis.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

The review covered five topics that the component depends on. The first two were differential privacy and its composition, and the auditing of private systems. The others were audit-log integrity, anonymity and singling out, and the regulations that the evidence must support. Most sources are peer-reviewed papers from security and privacy venues. Examples are the IEEE Symposium on Security and Privacy, the USENIX Security Symposium and ACM CCS. Standards and official guidance were used for the regulatory side. These include the GDPR, guidance from European data protection bodies, NIST frameworks and the HMAC and SHA-256 standards. Preprints were used for recent work on auditing tools. A source was included if it met one of three conditions. It defined a property that the component checks, or it described a way to check it. The third condition was that it showed a failure such a check could find.

2.2 Domain and theoretical foundations

Differential privacy. Dwork et al. [12], [13] defined differential privacy as a property of a randomized computation. Consider two datasets D and D′ that differ in one record, and any set of outputs S. A mechanism M satisfies ε-differential privacy if:

$$\Pr[M(D) \in S] \leq e^{\varepsilon} \cdot \Pr[M(D') \in S]$$

The Laplace mechanism answers a numerical query f by adding Laplace noise with scale $b = \Delta f / \varepsilon$. The sensitivity Δf is the largest change that one record can cause in the answer [12]. The noise has a mean of zero and a mean absolute value of b. This property is the basis of the reconstruction check in this thesis. When k mechanisms read the same records with budgets ε_1 to ε_k, their costs add up under sequential composition [14]:

$$\varepsilon_{total} = \varepsilon_1 + \varepsilon_2 + \ldots + \varepsilon_k$$

Deep learning uses the relaxed (ε, δ) form of the definition. Abadi et al. [15] introduced DP-SGD, and Mironov [16] introduced Rényi differential privacy (RDP) for tighter

accounting over many steps. In RDP accounting, the cumulative ε can only grow from one step to the next.

Message authentication. A hash function such as SHA-256 maps a file to a fixed digest [17]. A digest alone shows that a file changed, but anyone can compute a new one. HMAC adds a secret key to the hash, so only a key holder can produce a valid tag [18]. Comparing tags must take the same time for every input. Otherwise, timing differences can leak how many bytes matched [19].

Anonymity. Sweeney [20] defined k-anonymity. Each record must share its quasi-identifiers with at least $k - 1$ other records. Quasi-identifiers are attributes that are not unique alone but can identify a person in combination. The Article 29 Working Party named three risks that anonymization must address: singling out, linkability and inference [21]. Cohen and Nissim [22] gave a formal reading of singling out. They showed that k-anonymity alone does not always prevent it.

2.3 Related research

Auditing private systems. Several studies show that private software can fail while its reported ε looks correct. Mironov [23] showed that floating-point Laplace sampling can leak the true value through its least significant bits. Ding et al. [8] built a tool that searches for inputs where a mechanism breaks its claimed guarantee. Tramèr et al. [7] audited a published DP training method and found a bug that broke its claimed guarantee. Jagielski et al. [9] and Nasr et al. [10] used attacks to measure the real privacy of DP-SGD. These studies agree that a reported ε should be checked by someone other than the author of the code.

Reconstruction attacks. Dinur and Nissim [24] showed that answering too many queries with too little noise lets an attacker rebuild the private data. Their result explains why every query must be counted when budgets are added up. It also explains why the noise must have the size that the budget requires.

Tamper-evident logs. Schneier and Kelsey [25] designed audit logs whose past entries cannot be changed without detection. Crosby and Wallach [26] made such logs efficient with hash trees. In both works, the log is evidence for a third party as well as a record for the operator.

Re-identification and anonymity. Narayanan and Shmatikov [27] re-identified users in a sparse dataset by matching a few attributes. Rocher et al. [28] estimated that 15 demographic attributes would re-identify 99.98% of Americans. Machanavajjhala et al. [29] showed that k-anonymity can still reveal a sensitive value when a group shares it. These results show that metadata can identify records, even when the main data is protected.

Poisoning and secret leakage. Gu et al. [30] showed that a small number of poisoned training samples can plant a hidden behavior in a model. Meli et al. [31] studied public code repositories. They found that thousands of API keys and other secrets leak through them every day. Both problems sit outside the privacy mechanism, but both can undermine a privacy claim.

2.4 Existing systems products and techniques

Several tools and practices already address parts of the problem. Table 2.1 compares them with the needs of this component.

Table 2.1: Existing systems and techniques compared with the needs of the component

System or technique	What it provides	Limitation for this work
Opacus and similar DP libraries	DP-SGD with an RDP accountant and a reported ε [32]	The library reports its own ε; nothing checks the report afterwards
ML Privacy Meter	Membership inference audits of trained models [11]	Measures model leakage; does not check budget records, data integrity or regulatory mapping
DP testing tools	Search for inputs that break a mechanism's guarantee [8]	Test single mechanisms in isolation, not a whole pipeline's evidence
Tamper-evident logging	Logs whose entries cannot be changed without detection [25], [26]	Protect the log but do not judge what it says
Privacy management platforms and ISO/IEC 27701	Records of processing and management controls [33]	Document processes; do not run technical checks on the outputs of a pipeline
GDPR checklists	Lists of duties for a data controller [3], [34]	Not linked to evidence, so a ticked box shows nothing

Each tool covers one step. A DP library produces a budget, an auditing tool attacks a model, and a log protects records. None of them checks the evidence of a whole pipeline with separate code. None links each result to a legal duty.

2.5 Gap synthesis and conceptual framework

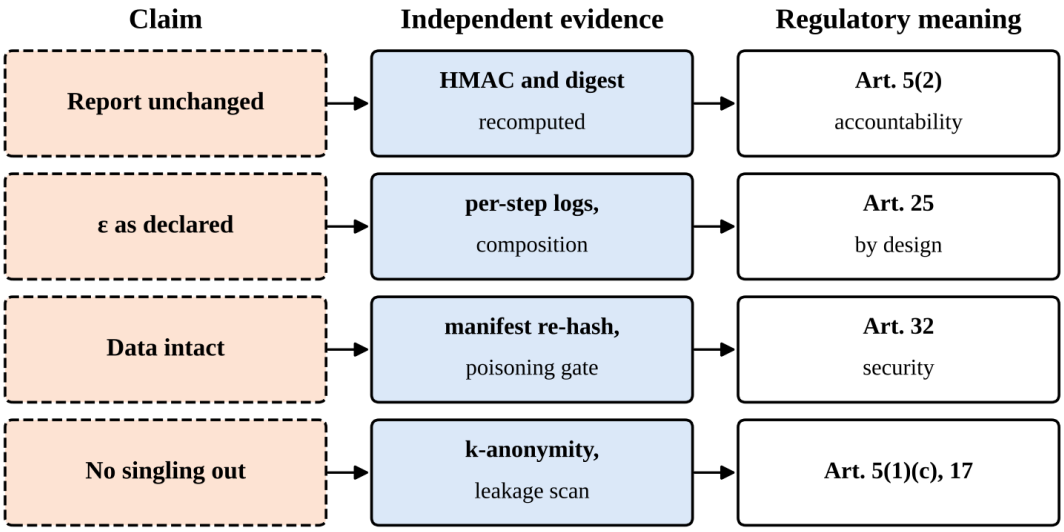
Table 2.2 summarizes the gaps that this component addresses and the approach taken for each.

Table 2.2: Research gaps addressed by the Risk and Compliance component

Gap	Limitation of existing work	Approach in this component
Budget records	Audits attack models but rarely check the reported ε against step-level evidence [9], [7]	Budget audit of every run against its per-step log
Calibration budget	Budgets of separate queries are not added up, and calibration is treated as configuration [4], [24]	Composition check and reconstruction attack on the calibration channel
Data around the model	Metadata and released files can identify records or leak secrets [28], [31]	k-anonymity, manifest re-hash and leakage scan
Evidence and law	Checklists are not linked to evidence, and reports can be edited [34], [25]	GDPR mapping marked verified only by a passing check; signed certificate

These gaps lead to the conceptual framework in Figure 2.1. Each claim of the pipeline is tested by an independent check. The check produces evidence, and the evidence supports a specific GDPR article. A provision counts as verified only when its check

has run and passed. The whole report is then fingerprinted and signed, so the verdict itself cannot be changed without detection.



a provision is marked verified only when the check behind it has run and passed

Figure 2.1: Conceptual framework of the Risk and Compliance component

3 RESEARCH METHODOLOGY

3.1 Research design

The research followed a design science approach with a quantitative evaluation. The component was first built as a working artifact. It was then tested in two ways. Unit tests fed it faults that were planted on purpose. The final run applied it to the real outputs of the group. The design had five parts:

- **Compliance baseline.** Before any check was written, GDPR articles were mapped to pipeline controls. Each control was given a check that would produce evidence for it.
- **Independence.** Each check was written apart from the code it tests. No signer, hash routine, key loader or noise function of another component was reused.
- **Known-fault validation.** Every check was tested with inputs that should fail, such as a changed report or a missing log. A check that cannot fail cannot give evidence.
- **Audit of the final outputs.** All checks were run on the outputs of the group's final experiment.
- **Feedback loop.** Faults were reported to the owning component. After a fix, the same unchanged check was run again.

Figure 3.1 shows the research process from the problem to the verified results.

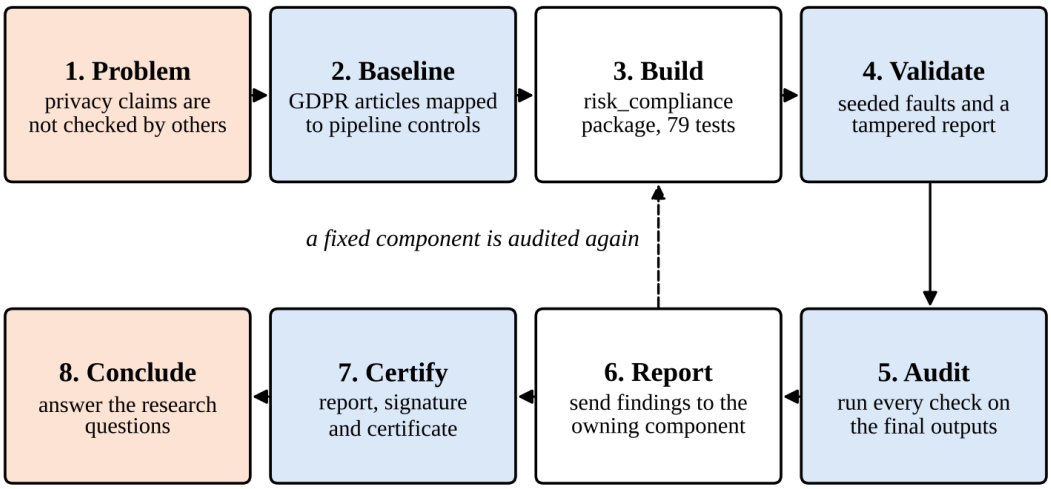


Figure 3.1: Research design and audit procedure of the component

3.2 Data participants or study materials

No human participants took part in this study. The study materials were the output files of the other three components. Table 3.1 lists them.

Table 3.1: Study materials received from the other components

Material	Producer	Content
audit_report.json and audit_certificate.json	Privacy Integration	Final ε, clipping norm, noise multiplier and steps for each run, with an HMAC-SHA256 tag and SHA-256 digest
budget_log.csv	Privacy Integration	Cumulative ε after every training step
calibration_stats.json	Privacy Integration	Seven privatized calibration statistics and the total ε spent
calibration_audit_log.json	Privacy Integration	Sensitivity, noise scale and noise drawn for each query; never published
Calibration source values	Privacy Integration	True statistics computed from the Waymo Open Dataset; never published
Encrypted towns and manifests	Dataset Generation	Five towns, 2,000 frames each, with a SHA-256 manifest per town
Frame metadata (CSV)	Dataset Generation	Weather, time of day, event type and town of each frame

Material	Producer	Content
poisoning_audit.json and utility_results.json	Utility Evaluation	Poisoning verdict and utility measures

The final experiment had four configurations, each trained with seeds 42, 43 and 44. These were a non-private baseline and three private configurations with target ε of 1, 3 and 8. This gave twelve signed audit reports. Each private run logged 3,090 training steps. The dataset held 10,000 frames in five CARLA towns. Each frame had four encrypted images: RGB, depth, semantic segmentation and a class mask. In total, the manifests listed 40,000 files.

Two calibration releases were checked. The earlier release ran each of the seven queries at $\varepsilon = 1.8$. The corrected release split a total of $\varepsilon = 1.8$ evenly over the seven queries. This gave $\varepsilon \approx 0.2571$ per query. The source values were computed from the Waymo Open Dataset [2]. The final dataset was captured from the corrected release.

3.3 Experimental or study procedure

The procedure had seven steps.

1. **Baseline.** The GDPR articles that apply to the pipeline were mapped to controls, owners and checks (Section 4.4).
2. **Signature check.** Each audit report was verified against its tag and digest with the shared key. Then the ε in a copy of one report was changed from 0.990572 to 0.01. The copy was then verified again.
3. **Budget audit.** For every run, the reported final ε was compared with the last row of its budget log. The log was also checked for any decrease and against the limit of $\varepsilon = 10$.
4. **Calibration check.** The noise log was reconciled with the published values. A reconstruction attack measured the error of the best estimate an attacker can make. The composition check added up the cost of all seven queries.
5. **Data checks.** Every decrypted file was hashed and compared with its manifest. The metadata was grouped by its quasi-identifiers to measure k. The release folder was scanned for leaked paths, addresses and secrets.

6. **Report.** The results were combined with the poisoning verdict of Utility Evaluation into a compliance report with an overall verdict.
7. **Certificate.** The report was fingerprinted with SHA-256, signed with HMAC-SHA256 and printed as a one-page certificate.

The calibration check was run twice. It was first run on the earlier release. The fault it found was reported to Privacy Integration, which fixed its calibration module. The same check was then run unchanged on the corrected release.

3.4 Measures and evaluation metrics

Table 3.2 lists the measures used in the evaluation.

Table 3.2: Measures and evaluation metrics

Measure	Definition	Pass condition
Signature validity	Recomputed HMAC-SHA256 tag and SHA-256 digest match the stored values	Both match for every report
Tamper detection	Verification of a deliberately changed copy	The copy is rejected
ε agreement	Absolute difference between reported ε and last logged ε	Below 10^{-4}
Monotonicity	No decrease in cumulative ε between steps	No decrease larger than 10^{-9}
Ceiling	Largest logged ε	At or below 10
Reconciliation	Source value plus logged noise equals published value	Equal to four decimals
Calibration ratio	Mean observed absolute error divided by mean expected error Δf / ε	Between 0.3 and 3.0
Composed ε	Sum of per-query ε recovered from the noise log	At or below the claimed total
Manifest match	Recomputed SHA-256 of each file against its manifest	No mismatched or missing file
k	Size of the smallest group of frames sharing all quasi-identifiers	At least 5
Leakage findings	Matches of seven sensitive patterns in released files	None, otherwise review

The calibration ratio needs a short explanation. Laplace noise has a mean of zero. As a result, an attacker can do no better than take each published value as its estimate. The expected absolute error of that estimate is the noise scale $b = \Delta f / \varepsilon$. A ratio near 1.0 means the noise had the size that the budget requires. A ratio far below 1.0 means too little noise, which would break the guarantee. The bounds of 0.3 and 3.0 are wide, because only five scalar fields are compared.

The threshold of $k = 5$ was set in the configuration file before the first run on real metadata.

3.5 Data analysis

Each check produced a JSON result with its own verdict. The compliance report then combined the results with three verdict rules. The overall verdict was PASS only when every check had run and passed. It was FAIL when any check failed. It was INCOMPLETE when a check had not run. An INCOMPLETE verdict is not a failure. It means the evidence is not yet complete. Each GDPR mapping took one of three states: verified, not verified, or not run. This kept a missing check from being read as a passing one.

The poisoning gate of Utility Evaluation can return FILTERED. This means it found suspicious batches and removed them before training. The report counts FILTERED as passing, because the control did its job. The leakage scan returns REVIEW REQUIRED when it finds a match, not FAIL. A match may be harmless, such as a relative path, and needs a person to judge it.

For the composition check, the per-query ε was recovered from the noise log as $\varepsilon = \Delta f / b$. All seven queries read the same records, so their costs were added under sequential composition [14].

3.6 Ethics privacy security and responsible AI

All frames in this work were generated inside the CARLA simulator, so no real person was recorded. The calibration source values, however, were computed from the Waymo Open Dataset [2]. They are aggregates, but they come from real road recordings. The source values and the calibration noise log can each reveal the source

statistics, so neither was ever published. The reconstruction result file holds the source values, so this thesis reports only aggregate results from it.

The HMAC key was loaded from a local .env file and was never written into source code. The key was received through a separate channel from the AES dataset key. The manifest check needs no key at all, because the manifests hold digests of the plaintext files. This kept the auditor from holding more secrets than its checks need.

The component does not claim legal compliance. Its report states that the verdict covers only the listed checks. The certificate states that it is not a legal certification of GDPR compliance. Known weaknesses are written into every report, so a reader sees the limits next to the verdict.

3.7 Reproducibility and research artefacts

All thresholds were kept in one file, config.yaml, and loaded into typed settings at start-up. Each check writes its result to a JSON file, so a later reader can inspect each verdict and its inputs. The same report always produces the same certificate PDF. A unit test checks this. The software was fixed at Python 3.11, with pandas, NumPy, PyCryptodome and ReportLab.

The research artefacts are the risk_compliance package and its nine command-line tools. Other artefacts are the result files of each check, the compliance report and the signed certificate. The certificate record holds the SHA-256 of the report and of the PDF. Any later change to either file can be detected. The earlier and corrected calibration results were both kept. This lets the fault and its fix be shown side by side.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The requirements came from three sources: the SRS of the group, the outputs of the other components and the stakeholders. The project proposal named four stakeholder groups. Autonomous vehicle developers need a privacy record they can show to others. Data protection officers and regulators need evidence they can check. Researchers want to share datasets safely, and road users need their data kept private. Table 4.1 lists the final functional requirements of the component.

Table 4.1: Functional requirements of the Risk and Compliance component

ID	Requirement	Source
RC-01	Verify every signed audit report with independent code, and reject a changed report	Accountability, GDPR Art. 5(2)
RC-02	Compare each reported ε with its per-step log, and check monotonicity and the limit	Proposal limit of ε = 10
RC-03	Reconcile the calibration noise log and measure reconstruction error	Calibration privacy
RC-04	Check that the calibration queries compose to the reported total ε	Sequential composition [14]
RC-05	Re-hash every decrypted file against its manifest and stop the pipeline on a mismatch	Security of processing, Art. 32
RC-06	Measure k-anonymity of the released frame metadata	Singling out [21]
RC-07	Scan released files for paths, addresses, coordinates and credentials	Data minimization, Art. 5(1)(c)
RC-08	Map each GDPR provision to a control, an owner and a check	SRS FR-23
RC-09	Combine all results into one report with PASS, FAIL or INCOMPLETE and declared weaknesses	SRS FR-24
RC-10	Issue a certificate whose fingerprint and signature can be checked by a command	Accountability, Art. 5(2)

These requirements differ from the SRS. Table 4.2 traces each original requirement to its final state.

Table 4.2: Trace from the SRS requirements to the final design

SRS ID	Original requirement	Final state
FR-20	Shadow-model membership inference tests on each ε configuration	Run by Privacy Integration on its own models; an independent repeat is planned future work
FR-21	Re-identification attack on synthetic trajectories	Not implemented; no individual real records exist in the final pipeline
FR-22	Standard results table of all attacks	Replaced by per-check JSON results and the compliance report
FR-23	GDPR checklist mapping controls to articles	Implemented as eight mappings, each linked to a check (RC-08)
FR-24	Integrated compliance report	Implemented as the compliance report and signed certificate (RC-09, RC-10)

The non-functional requirements were as follows:

- **Independence.** No check imports the signing, hashing, key or noise code of the component it tests.
- **Least privilege.** The component holds only the HMAC key. It needs no AES key and no access to trained models.
- **Honest verdicts.** A check that has not run is never reported as passed. Every report states what it does not cover.
- **Determinism.** The same inputs give the same results, and the same report gives the same certificate PDF.
- **Interoperability.** Every exchanged file follows a JSON Schema contract of the group.
- **Low cost.** All checks run on a laptop CPU in seconds, without a GPU.

4.2 Component architecture within the complete solution

Figure 4.1 shows where the component sits in the group pipeline. It reads the evidence of all three other components. Its first check, the manifest check, runs right after decryption. If that check fails, the pipeline stops and nothing downstream may use the

data. The other checks run after training and utility evaluation are complete. The component then writes the compliance report and the certificate.

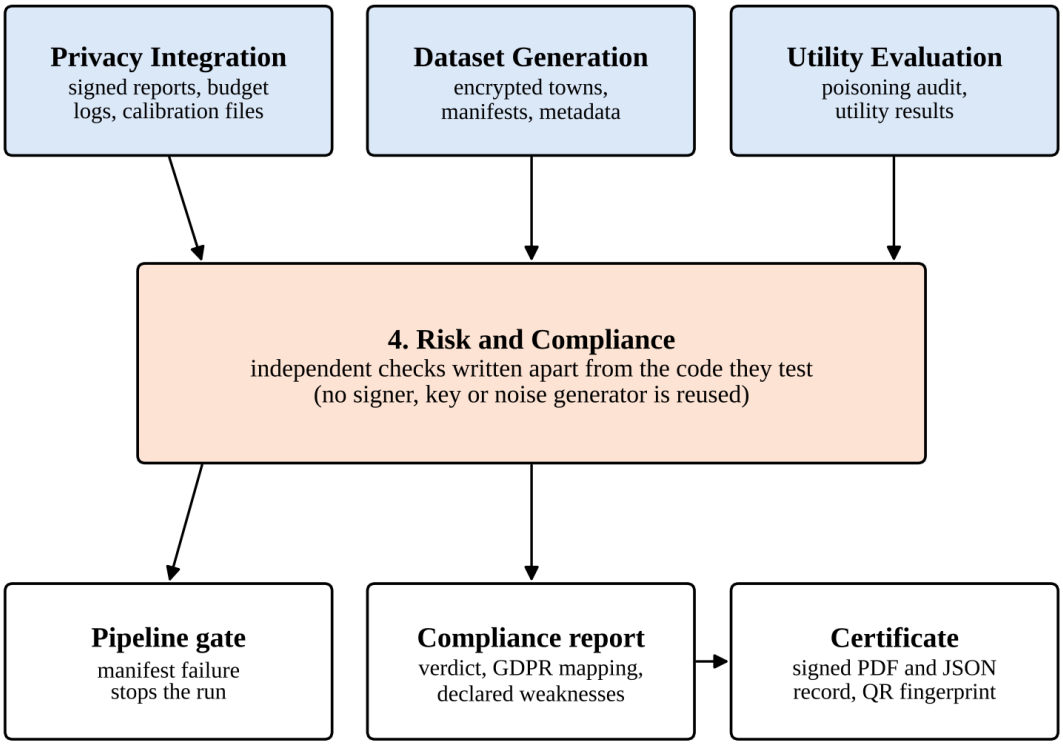


Figure 4.1: Position of Risk and Compliance in the group pipeline

This position gives the component a clear boundary. It reads the outputs of the other components but never changes them. It sends findings back to their owners and runs the same check again after a fix. The only file it releases is the compliance report with its certificate.

4.3 Design alternatives and decisions

Table 4.3 lists the main design choices, the alternatives considered and the reason for each choice.

Table 4.3: Design alternatives and decisions

Decision	Alternative	Reason for the choice
Write each verifier from scratch	Call the producer's own verify function	A defect in the producer's code would then pass unnoticed

Decision	Alternative	Reason for the choice
Constant-time tag comparison	Plain string comparison	Avoids leaking how many bytes matched through timing [19]
Treat the non-private run as a declaration	Treat it as ε = 0	A run without DP has no budget; ε = 0 would claim perfect privacy
Recover per-query ε from the noise scale	Trust the ε field in the log	The field name in early logs held the noise scale, not ε
Three verdicts with INCOMPLETE	PASS or FAIL only	A missing check must not look like a pass or a failure
Count FILTERED as passing	Count it as a failure	The poisoning gate worked by removing the suspicious batches
Leakage findings need review	Fail on any match	Patterns can match harmless text, so a person must judge them
Sign the SHA-256 of the report bytes	Sign the PDF only	The JSON report is the evidence; the PDF is a readable copy
HMAC-SHA256 for the certificate	Public-key signature	Matches the keys already used by the group; public-key signatures are future work
Verify manifests without the key	Decrypt inside this component	The manifests hold plaintext digests, so the auditor never needs the AES key

The first decision shaped the rest of the design. An auditor that reuses the code it audits can only confirm that the code agrees with itself. For this reason, every verifier in this component was written from the published file formats alone.

4.4 Detailed design

The component is an installable Python package called risk_compliance. It has ten modules and nine command-line tools. Figure 4.2 shows the checks, the evidence each one reads and its result on the final outputs.

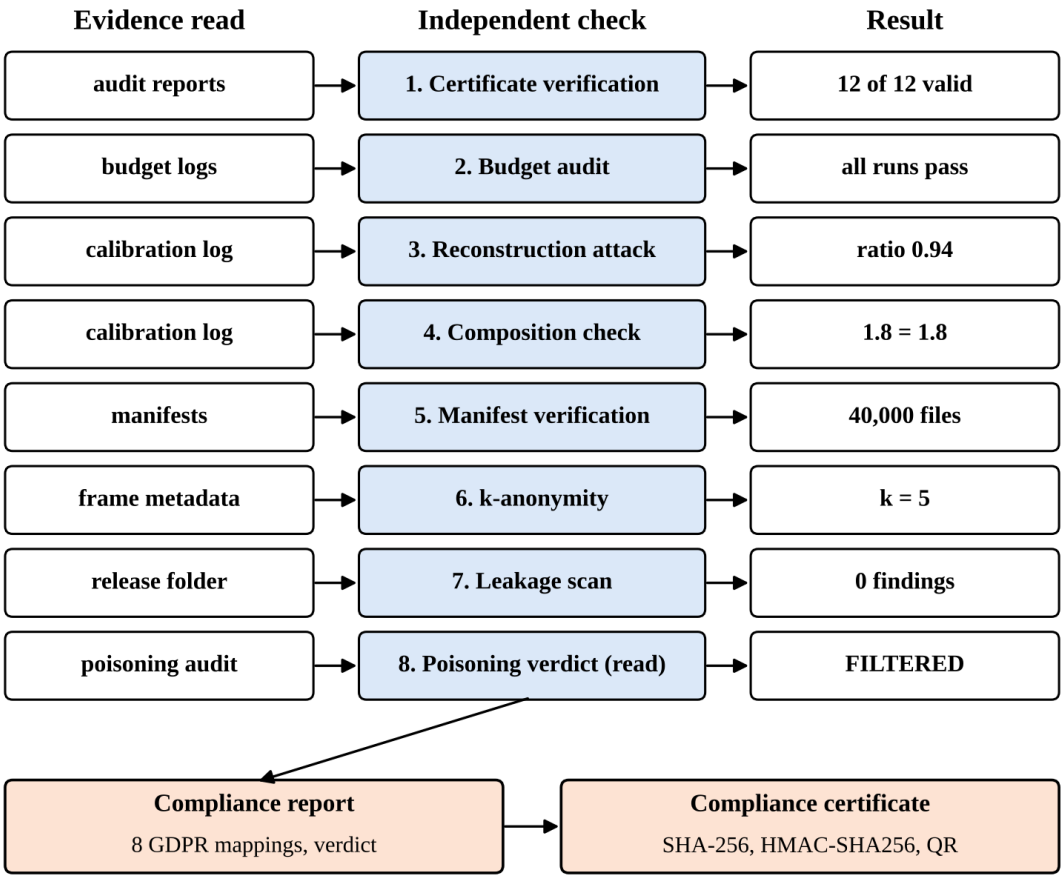


Figure 4.2: Checks of the Risk and Compliance component, their inputs and their results on the final outputs

Settings module. This module loads config.yaml into typed settings. It holds the ε limit of 10 and the agreement tolerance of 10^{-4}. It also holds the sensitivities, the quasi-identifiers and the k threshold. A missing or invalid key raises a configuration error at start-up.

Verification module. For each run, it reads the report as text and computes its HMAC-SHA256 tag and SHA-256 digest. Both values are compared with the certificate in constant time. The tamper test copies one report and its certificate to a work folder. It changes one field and verifies the copy. The original files are never touched.

Budget audit module. This module reads each audit report and its budget log. It compares the reported final ε with the last logged value and checks that ε never decreases. It also checks that the highest value stays within the limit. A private run without a log fails, because its ε claim has no evidence. A run that declares no DP is recorded as a declaration with no budget.

Reconstruction module. For each scalar field, it computes the absolute error between the published and source values. It compares the mean error with the mean expected error Δf / ε. It also checks that each source value plus its logged noise gives the published value. The histogram queries carry no single noise value, so they are skipped in this step.

Composition module. This module reads each query in the noise log and recovers the ε it spent. The recovered values are added and compared with the total that the calibration file reports. If the sum is larger, the result states the per-query ε that would meet the claim.

Manifest module. It hashes every decrypted file in 64 KB chunks and compares the digest with the manifest entry. Files are counted as verified, mismatched or missing.

Anonymity module. It joins all metadata files and groups the frames by weather, time of day, event type and town. The smallest group size is k. The module also counts groups of size one and groups below the threshold.

Leakage module. It searches every text file of the release for seven patterns. These are Windows and Unix absolute paths, email addresses, IP addresses, GPS coordinates, credential-like strings and machine names.

Mapping module. This module holds the GDPR mapping and the declared weaknesses. Table 4.4 shows the eight mappings.

Table 4.4: GDPR provisions mapped to controls, owners and evidence checks

Provision	Requirement	Control	Evidence check
Art. 5(1)(b)	Purpose limitation	Real statistics used only to calibrate the simulator, never for training	Calibration audit log
Art. 5(1)(c)	Data minimization	Only seven aggregate fields extracted	Calibration audit log
Art. 5(2)	Accountability	HMAC-SHA256 signed audit reports	Certificate verification
Art. 17	Right to erasure	No individual record stored in the pipeline	k-anonymity
Art. 25	Data protection by design	DP applied during calibration and training	Budget audit
Art. 25	Budget accounting at calibration	Laplace noise with the total ε reported	Calibration composition
Art. 32	Security of processing	AES-256-CBC, SHA-256 and role-based access control	Manifest verification
Art. 32	Integrity of processed data	Poisoning detection before training	Poisoning detection

Report builder. This tool loads every result file and the poisoning verdict of Utility Evaluation. It marks each mapping as verified, not verified or not run. It then applies the verdict rules of Section 3.5. The report also records the utility evidence, the thresholds and the declared weaknesses. A fixed note states that the verdict covers only the listed checks.

Certificate module. This module computes the SHA-256 of the exact bytes of compliance_report.json. It signs this fingerprint with HMAC-SHA256 and draws a one-page A4 PDF with ReportLab. The page shows the verdict, each check, the weaknesses, the fingerprint and the signature. A QR code encodes the fingerprint, which is also stored in the PDF metadata. A JSON record holds both values and the SHA-256 of the PDF. The verify command recomputes these values and fails if anything changed. It also checks that the PDF still embeds the fingerprint. Without a

key, the certificate states that it is unsigned. One signature line is left blank for a handwritten signature.

4.5 Standards and professional practice

The design follows the standards and practices listed below.

- **GDPR.** The mapping covers Articles 5, 17, 25 and 32 [3]. The reading of Article 25 follows the guidelines of the European Data Protection Board [34].
- **Anonymization guidance.** The k-anonymity and leakage checks address the singling-out risk named in Opinion 05/2014 [21].
- **Risk frameworks.** The checks give evidence for the Govern and Protect functions of the NIST Privacy Framework [35]. The mapping itself covers only the GDPR.
- **Privacy management.** The declared weaknesses and owners follow the record-keeping practice of ISO/IEC 27701 [33].
- **Cryptographic standards.** Digests use SHA-256 from FIPS 180-4 [17], and tags use HMAC as defined in RFC 2104 [18].
- **Engineering practice.** Keys were kept out of version control. Every check has tests for its failure cases. All tests run on each push through continuous integration.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

The component was developed and run on a laptop with Python 3.11. Its checks need no GPU. Table 5.1 lists the main tools and their purpose.

Table 5.1: Development environment and technology stack

Tool	Purpose
Python 3.11	Main programming language
Python hmac and hashlib	HMAC-SHA256 tags, SHA-256 digests and constant-time comparison
pandas	Reading budget logs and grouping metadata for k-anonymity
NumPy and SciPy	Numerical work in the calibration checks
PyYAML	Loading the configuration file
PyCryptodome	Cryptographic support shared with the group
ReportLab	One-page certificate PDF with a QR code of the fingerprint
JSON Schema (group contracts)	Checking every result file against its contract
pytest and GitHub Actions	79 unit tests and continuous integration
Git and GitHub	Version control in the group repository

5.2 Component implementation

Each module has a command-line entry point, so every check can be run on its own. Table 5.2 lists the nine tools. Each tool writes its result as JSON to the outputs folder unless another path is given.

Table 5.2: Command-line tools of the component

Command	Function
rc-verify-certs	Verify every signed audit report and run the tamper test
rc-audit-budgets	Compare reported and logged ε for every run
rc-reconstruct	Reconstruction attack and noise log reconciliation
rc-calibration-composition	Add up the calibration budget over all queries
rc-verify-manifest	Re-hash the decrypted dataset against the manifests
rc-anonymity	Measure k-anonymity of the frame metadata

Command	Function
rc-scan-leakage	Scan the release folder for sensitive patterns
rc-report	Build the compliance report and overall verdict
rc-certificate	Build the certificate, or check it with --verify

Verification and tamper test. The verify function reads the report as UTF-8 text and signs the exact text. It uses hmac.compare_digest for both the tag and the digest. A unit test confirms that even a whitespace change is detected. Another test confirms that a wrong key fails the tag but still matches the digest. This shows that the digest alone cannot prove who wrote a report. The tamper test edits the field results.final_epsilon in a copy. It returns the original value with the result, so the report can show what was changed.

Budget audit. The audit reads budget_log.csv with pandas and takes the cumulative_eps column. Monotonicity is tested on the differences between steps, with a tolerance of 10^{-9} for rounding. A final ε slightly above the target is noted but not failed. RDP accounting is evaluated between steps, so a small overshoot can occur. The limit of 10 is still enforced.

Calibration checks. The reconstruction tool reads three files: the source values, the published statistics and the noise log. The source values are read from a path given at run time. They are never copied into the outputs of the group. The composition tool reads only the noise log and the published file. It supports three log formats, which matter for the challenges below.

Report and certificate. The report builder reads the result files listed in Table 5.3. For each one, it reads a single key that holds the verdict. The certificate module draws a seal that shows the verdict, a table of checks and a fingerprint box. It also adds a QR

code and a footer with the verify command. It draws the PDF in the invariant mode of ReportLab. As a result, the same report and issue time always give the same file.

Table 5.3: Checks combined in the compliance report

Check	Result file	Verdict key
Certificate verification	certificate_verification.json	all_valid
Budget audit	budget_audit.json	all_passed
Calibration audit log	reconstruction_attack.json	audit_log_reconciliation
Calibration composition	calibration_composition.json	verdict
k-anonymity	anonymity.json	verdict
Manifest verification	manifest_verification.json	all_valid
Poisoning detection	poisoning_audit.json (Utility Evaluation)	verdict

5.2.1 Implementation challenges

Ambiguous budget field. The first noise logs had a field called epsilon_used. For each query, it held the Laplace scale $\Delta f / \varepsilon$, not ε. A check that trusted the field name would have read 0.556 as the budget of a query that spent 1.8. For this reason, the composition module tests what the field holds. If every value equals Δf divided by the stated ε, the field holds the scale. The current logs record epsilon_spent and laplace_scale explicitly. The module prefers these fields when they exist. Unit tests cover all three formats.

Coordinates in JSON. A simple GPS pattern would match two decimals separated by a comma and a line break. In JSON result files, lists of numbers would then be reported as coordinates. To avoid this, the pattern was limited to a single line. A test checks that a list of numbers does not match.

Paths in mounted folders. When the pipeline runs in a container, files live under folders such as /tmp and /mnt. The Unix path pattern covers these folders as well as home folders, and a test checks both.

The non-private baseline. The baseline run has no budget log, because it applies no DP. A plain audit would fail it. Treating it as $\varepsilon = 0$ would also be wrong, because that

claims perfect privacy. Instead, the audit reads the flag differential_privacy_applied. When the flag is false, it records the run as a declaration and gives no ε.

Column names in metadata. The first k-anonymity check failed, because the CSV column names did not match the expected names. The quasi-identifiers in config.yaml were set to the names that Dataset Generation uses. A missing column now raises an error. Without this, a missing column would reduce the number of quasi-identifiers and raise k without warning.

5.3 Integration with the other components

The group pipeline runs each component as a list of stages, and seven of them belong to this component. Each stage names its inputs, its outputs and the stages it needs. The manifest stage has a gate. If any file fails, the stage stops the run with a message that nothing downstream may use the data. Each result file is checked against its JSON Schema contract as soon as it is written. Table 5.4 lists the files that the component exchanges.

Table 5.4: Files exchanged by the Risk and Compliance component

File	Direction	Purpose
audit_report.json and audit_certificate.json	From Privacy Integration	Signed evidence for each training run
budget_log.csv	From Privacy Integration	Per-step ε for the budget audit
calibration_stats.json	From Privacy Integration	Published statistics and claimed total ε
calibration_audit_log.json	From Privacy Integration only	Noise per query; never published
Manifests and metadata	From Dataset Generation	Digests for the manifest check and quasi-identifiers
Decrypted towns	From Utility Evaluation (decrypted once)	Files to re-hash
poisoning_audit.json and utility_results.json	From Utility Evaluation	Poisoning verdict and utility evidence
manifest_verification.json	To the pipeline gate	Decides whether the run may continue
compliance_report.json and certificate	To the release folder	Verdict, GDPR mapping and signed record

The pipeline decrypts the dataset once. This is a step of Utility Evaluation. The manifest stage then hashes the decrypted files, so the auditor never needs the AES key. The HMAC key reached this component through a separate channel from the AES key.

5.4 Verification and software testing

The component was tested at three levels. Unit tests checked each module on its own with small fixtures. Each check has at least one test where it must fail. Examples are a changed report, a broken budget log, falsified noise, an altered file and a unique metadata row. Table 5.5 lists the tests by module. All 79 tests passed.

Table 5.5: Unit tests by module

Module	Tests	Main failure cases covered
Verification	10	Changed report, whitespace change, wrong key, original files untouched
Budget audit	11	Ceiling breach, report and log disagree, missing log, baseline is not $\varepsilon = 0$
Reconstruction	7	Under-noising flagged, falsified noise detected, no matching fields
Composition	6	Seven queries at full budget compose to seven times the claim
Manifest	4	Altered file, missing file, missing manifest
k-anonymity	8	Unique combination gives $k = 1$, missing column raises an error
Leakage	13	Paths, email, IP, coordinates, credentials; number lists not flagged
Mapping	9	Unverified state differs from failed; calibration fix recorded
Compliance report	5	Composition failure fails the report; missing evidence is INCOMPLETE
Certificate	6	Verify catches every kind of change; same report gives same PDF

Shared library tests checked the JSON contracts between the components. An end-to-end test ran the whole pipeline on a small fixture dataset. GitHub Actions ran the tests of every component and the end-to-end test on each push.

The functional requirements were then checked against the final outputs. Table 5.6 lists how each requirement was verified.

Table 5.6: Verification of the functional requirements

ID	Requirement	Verification	Status
RC-01	Verify reports, reject changes	12 of 12 valid; changed copy rejected	Passed
RC-02	Audit ε against logs	9 private runs agree; 3 baselines declared	Passed
RC-03	Reconcile log, measure error	5 of 5 fields reconcile; ratio 0.94	Passed
RC-04	Check composition	Earlier release failed at 12.6; corrected release 1.8	Passed
RC-05	Re-hash and gate	40,000 of 40,000 files match	Passed
RC-06	k-anonymity	k = 5, no unique rows	Passed
RC-07	Leakage scan	9 files, 0 findings	Passed
RC-08	GDPR mapping	8 of 8 provisions verified	Passed
RC-09	Report with verdicts	Overall PASS, 2 weaknesses declared	Passed
RC-10	Signed certificate	Issued; report and PDF match the record	Passed

5.5 Deployment and operation

The component is installed as a Python package next to the shared library of the group. In the intended deployment, it runs on a machine separate from the producing components. That machine holds the HMAC key and receives the evidence files. The noise log and the source values travel to it through a separate channel from the published release. The key is supplied through an environment variable or a local .env file. This lets the same code run on any machine without changes.

In the group pipeline, the stages run in order without manual steps. A reviewer can also run each command by hand on a folder of evidence. The certificate can be checked later by anyone with the report, the PDF and the key. The command is rc-certificate --verify. It reports four results: the report fingerprint, the signature, the PDF digest and the fingerprint inside the PDF.

6 RESULTS AND VALIDATION

6.1 Research results

6.1.1 Integrity of the signed evidence

The certificate check found twelve audit reports under the runs folder. These were the four configurations trained with three seeds each. Table 6.1 gives the result. Every report had a valid HMAC-SHA256 tag and a valid SHA-256 digest.

Table 6.1: Verification of the signed audit reports

Measure	Result
Reports found	12 (4 configurations × 3 seeds)
Valid HMAC-SHA256 tags	12 of 12
Valid SHA-256 digests	12 of 12
Tamper test target	seed_42/config_a
Change made	Final ε 0.990572 replaced with 0.01
Changed copy accepted	No; tag and digest both failed

The tamper test shows why the signature matters. Changing ε from 0.990572 to 0.01 would make a weak budget look about 100 times stronger. Such a change cannot be seen in the file itself. The check rejected the changed copy, and the original report was left untouched. Without the key, an editor of the report could recompute the digest but not the tag.

6.1.2 Privacy budget audit

The budget audit compared each reported ε with the per-step log of the same run. Table 6.2 gives the result for seed 42. The other two seeds gave identical values, because the target ε and the number of steps were the same.

All nine private runs passed. In each run, the reported ε matched the last logged value exactly, and ε never fell between steps. The highest value of 7.995 stayed well below the limit of 10, and no run stopped early. Each final ε was slightly below its target, which is expected. Opacus chooses the noise so that the budget at the last step does not exceed the target. The three baseline runs declared that no DP was applied. The audit recorded them as declarations with no ε, not as ε = 0. Figure 6.1 compares the targets with the logged values.

Table 6.2: Budget audit of each configuration, with all values in ε (seed 42; seeds 43 and 44 identical)

Run	Target	Reported	Logged final	Steps	Never falls	Limit	Verdict
Config A	1.0	0.990572	0.990572	3,090	Yes	Kept	PASS
Config B	3.0	2.99984	2.99984	3,090	Yes	Kept	PASS
Config C	8.0	7.995129	7.995129	3,090	Yes	Kept	PASS
No DP	none	none	none	0	n/a	n/a	Declared

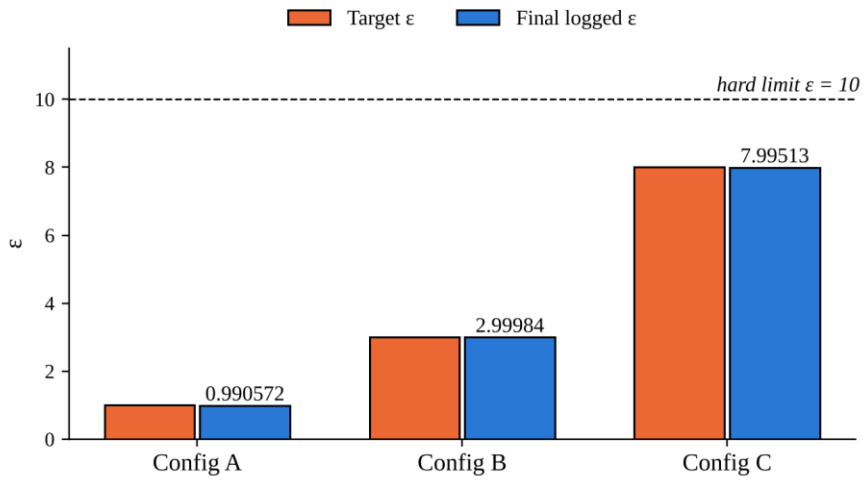


Figure 6.1: Target and final logged ε of the three private configurations

6.1.3 Calibration channel

The calibration channel was checked in three ways. Table 6.3 summarizes the results for both releases.

Table 6.3: Calibration checks on the earlier and corrected releases

Measure	Earlier release	Corrected release
ε per query	1.8	0.2571
Claimed total ε	1.8	1.8
Composed total ε (seven queries)	12.6	1.8
Composition verdict	FAIL	PASS
Noise log reconciliation	5 of 5 fields	5 of 5 fields
Mean observed reconstruction error	0.62	4.37
Mean expected error (Δf / ε)	0.67	4.67
Calibration ratio	0.94	0.94

Noise log reconciliation. For every scalar field of both releases, the source value plus the logged noise gave the published value. This shows that the noise log is an accurate record of the noise that was applied. This matters, because the reconstruction and composition checks both read the log.

Composition. On the earlier release, the check recovered ε = 1.8 for each of the seven queries. Under sequential composition, they add up to 12.6. The calibration file, however, reported a total of 1.8. The check returned FAIL and stated the fix: a per-query ε of 1.8 / 7 ≈ 0.2571. The finding was sent to Privacy Integration, which changed its calibration module to split a total budget over the queries. The same check was then run unchanged on the corrected release. It recovered ε ≈ 0.2571 per query and a total of exactly 1.8. Figure 6.2 shows both results.

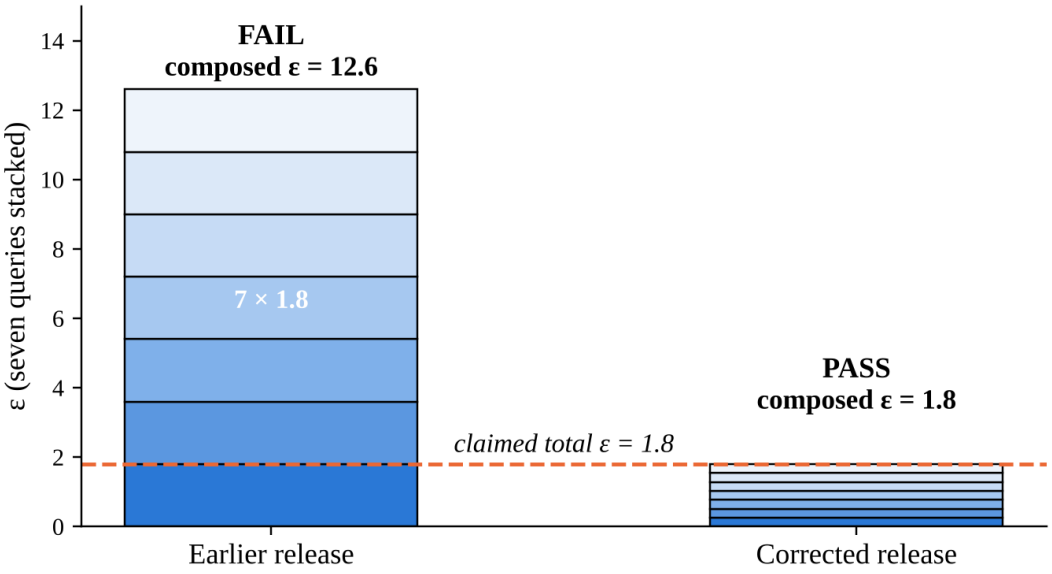


Figure 6.2: Composed calibration budget of the earlier and corrected releases

Reconstruction. An attacker who sees the published file can do no better than take each value as the estimate. On the corrected release, the mean absolute error of this estimate over the five scalar fields was 4.37. A correct Laplace mechanism at $\varepsilon \approx 0.2571$ gives an expected error of 4.67. The ratio of 0.94 is close to the ideal of 1.0. This means the noise had the size that the budget requires. The ratios of single fields ranged from 0.13 to 1.67. A wide spread is expected with one draw per field. Figure 6.3 compares the two releases.

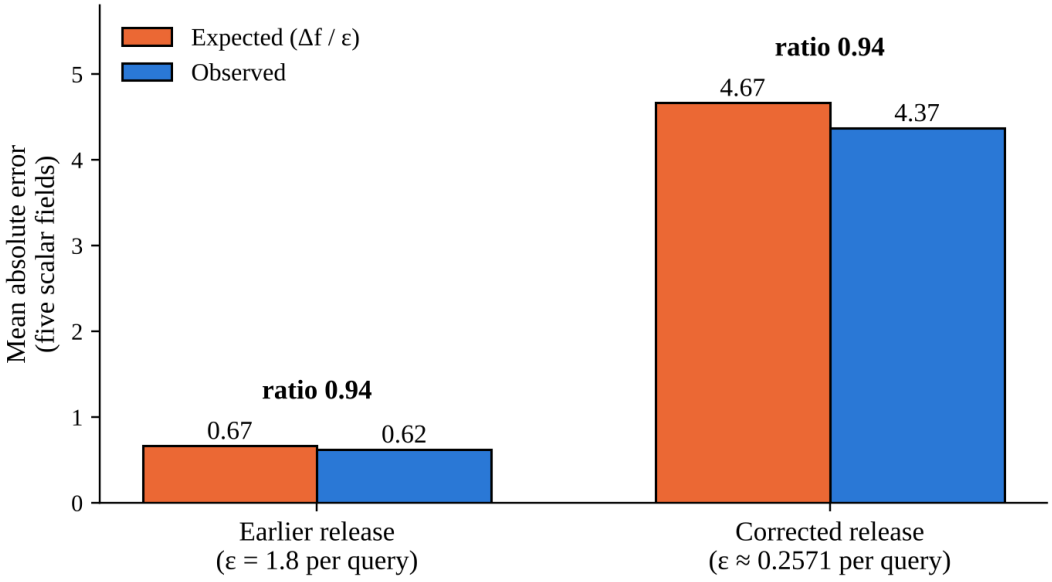


Figure 6.3: Observed and expected reconstruction error of the two releases

The earlier release also had a ratio of 0.94. Its noise was correct for each query on its own. The fault was only in how the per-query budgets were added up. A per-query check alone would have passed the earlier release. Only the composition check found the fault.

6.1.4 Dataset integrity and metadata

Table 6.4 gives the results of the data checks.

Table 6.4: Dataset integrity, anonymity and leakage results

Check	Result
Towns with a manifest	5 of 5
Files verified against the manifests	40,000 of 40,000 (8,000 per town)
Mismatched or missing files	0
Metadata rows	10,000
Quasi-identifiers	Weather, time of day, event type, town
Distinct combinations	192
Smallest group size (k)	5 (threshold 5)
Unique rows and groups below threshold	0 and 0
Release files scanned for leakage	9
Leakage findings	0

Every decrypted file matched its manifest, so the frames used for training were exactly those that Dataset Generation produced. The 10,000 metadata rows fell into 192 combinations of the four quasi-identifiers. The smallest combination held five frames. This means no frame could be singled out from its metadata alone. However, k equals the threshold exactly, so the margin is small. The leakage scan found no absolute paths, addresses, coordinates, credentials or machine names in the nine release files.

6.1.5 Poisoning verdict

The compliance report also read the poisoning audit of Utility Evaluation. The gate scanned 10,000 frames and 15,084 boxes. It found no structural faults but rejected 24 batches, which removed 2,400 frames. Its verdict was FILTERED, which the report counts as passing. Figure 6.4 shows the frames kept and removed in each town.

Town01 was also flagged as an outlier, because its cyclist share was 0.030 against 0.116 elsewhere.



Figure 6.4: Frames accepted and excluded by the poisoning gate in each town

6.2 Product and technical results

The final compliance report ran all seven checks, and none was missing. Table 6.5 shows the mapping states and the overall verdict.

Table 6.5: Final compliance report

Provision	Requirement	Evidence check	State
Art. 5(1)(b)	Purpose limitation	Calibration audit log	Verified
Art. 5(1)(c)	Data minimization	Calibration audit log	Verified
Art. 5(2)	Accountability	Certificate verification	Verified
Art. 17	Right to erasure	k-anonymity	Verified
Art. 25	Data protection by design	Budget audit	Verified
Art. 25	Calibration budget accounting	Calibration composition	Verified
Art. 32	Security of processing	Manifest verification	Verified
Art. 32	Integrity of processed data	Poisoning detection	Verified
Overall	Seven checks run, none missing		PASS

The report declared two weaknesses. The first is of medium severity. Opacus secure mode was turned off, so the DP-SGD noise came from a standard pseudo-random generator. The second is of low severity and is now fixed. The earlier calibration releases understated the total ε. The report adds a condition. The dataset is covered by the 1.8 total only once it is captured again from the corrected statistics. The dataset has since been captured again, so this condition is now met. The report also recorded the utility evidence of Utility Evaluation. This was a retention of 6.2% on mAP50-95 and an FID of 115.99.

The certificate was issued on 20 October 2026 with the identifier R26-RC-F3335EAE8286. Figure 6.5 shows it. The page carries the overall verdict and the result of each check. It also shows the SHA-256 of the report, the HMAC-SHA256 signature and a QR code of the fingerprint. The verify command checked the report fingerprint, the PDF digest and the fingerprint inside the PDF. All three matched the record. The signature check needs the HMAC key, which is held outside the repository. The signature line was left blank for a handwritten signature.



Figure 6.5: Compliance certificate issued for the final outputs

6.3 User or stakeholder validation

No formal user study was carried out. The component was validated by the team members who used its results. The composition finding was the main case. Privacy Integration accepted the finding and changed its calibration module. It recorded the fix in the declared limitations of its own privacy certificate. That entry states that the fault

was found by the composition check of this component. Privacy Integration also added tests for the composition of its calibration budget.

The other components used the results in the same way. The manifest result told Utility Evaluation and Privacy Integration that the decrypted frames matched what Dataset Generation had produced. The scope change from the SRS is reported openly in Sections 1.7 and 4.1. This lets readers see which planned attacks were not carried out.

6.4 Integration results

The end-to-end test of the group ran the whole pipeline on a small fixture dataset. In this test, the seven stages of the component ran in order. The manifest gate passed, so the training and evaluation stages could start. Each result file with a contract passed its JSON Schema check. The report stage waited for the poisoning audit and the utility results before building the verdict. The leakage stage ran last, on the assembled release folder. The results in Section 6.1 were produced with the same commands on the outputs of the final experiment.

The tests also covered the verdict rules. One test builds the report without the utility evidence and confirms that the verdict is INCOMPLETE, not PASS. Another test confirms that a failed composition check makes the whole report FAIL.

6.5 Summary of findings

Table 6.6 summarizes the findings against the research questions.

Table 6.6: Summary of findings

RQ	Finding	Evidence
RQ1	Signed evidence can be verified without the producer's code	12 of 12 reports valid; tamper detected; ε agrees in 9 of 9 private runs
RQ2	The earlier calibration release overspent; the corrected one spends what it claims, with noise of the right size	Composed ε 12.6 vs 1.8, then 1.8; ratio 0.94
RQ3	The dataset and its metadata kept their integrity, with no singling out and no leakage	40,000 files match; k = 5; 0 findings
RQ4	The results support a verified GDPR mapping and a certificate that shows any change	8 of 8 provisions verified; PASS; signed certificate

A later review of the calibration code found one more weakness. The calibration noise was drawn from a pseudo-random generator with a fixed seed of 42. This seed was stored in the public configuration file of Privacy Integration. With the seed, the noise of every query could be generated again and subtracted from the published values. This would have revealed the source statistics exactly. The reconstruction check assumes that the attacker does not know the noise, so it could not detect this weakness. Both releases had the same ratio of 0.94. This fits the same random draws scaled by a different noise scale. Privacy Integration corrected the fault before the dataset was captured again. Chapter 7 discusses this result.

7 DISCUSSION

7.1 Interpretation of findings

The results give a positive answer to the research question, within clear limits. For RQ1, every signed report and every budget log could be checked without the producer's code. The check also rejected a changed report. For RQ2, the composition check found a real fault, and the same check confirmed the fix. For RQ3, the dataset matched its manifests, and its metadata did not allow singling out. For RQ4, the results supported all eight GDPR mappings and a certificate that shows any later change.

The most useful result was the failed composition check. The earlier calibration release spent $\varepsilon = 12.6$ and reported 1.8. The figure of 1.8 had appeared in earlier outputs of the project without question. Each query added noise of the correct size, and the reconstruction ratio of 0.94 confirmed this. The fault was in the accounting across queries. A check that looked at one query at a time could not see it. This supports the working hypothesis. A check written apart from the producing code found a fault that the producer had missed.

The passing results need a careful reading too. A PASS means the evidence agrees with the claims that were checked. It does not mean the pipeline is private in every sense. For example, the budget audit confirms that ε was logged and reported correctly. It does not confirm that Opacus computed ε correctly. Likewise, $k = 5$ shows that metadata alone cannot single out a frame. It says nothing about what the images themselves reveal.

The fixed seed shows the limit of the reconstruction check most clearly. The check measured the right thing: the noise had the size that $\varepsilon = 0.2571$ requires. However, it assumed that the attacker cannot know the noise. The noise was drawn with a fixed seed of 42 that was stored in a public file. Anyone with the repository could draw the same noise and remove it. Until the fault was corrected, the formal guarantee did not hold against such an attacker. No check in the component looks for this kind of fault. It was found by reading the calibration code, not by running the checks. For this reason, a check for secure noise sources is the first item of future work.

7.2 Comparison with literature baselines and existing solutions

The composition fault matches the warning of Dinur and Nissim [24]. Privacy is lost through the total of many answers, not through any single answer. Here, seven correctly noised answers together spent seven times the reported budget. The fault also matches the pattern found by Tramèr et al. [7] and Ding et al. [8]. Private software can report a correct-looking ε while the real guarantee is weaker. These studies found faults by attacking or testing the mechanism. This component found a fault by checking the accounting records, which is a cheaper method.

The fixed seed is a known type of fault. Mironov [23] showed that the way noise is generated can break a guarantee that is correct on paper. A predictable random source is an extreme case of the same problem. It also matches the declared weakness of Privacy Integration about Opacus secure mode. Both faults come from the random number generator, not from the privacy formula.

The tamper test agrees with the design of tamper-evident logs [25], [26]. A digest alone did not prove who wrote a report. In the unit test, a wrong key still matched the digest. The keyed tag did. The k-anonymity result should be read with the warnings of Machanavajjhala et al. [29] and Cohen and Nissim [22]. A k of 5 rules out the simplest form of singling out but not every form.

Compared with existing tools, the component fills the gap described in Section 2.4. ML Privacy Meter [11] attacks models but does not check budget records, data integrity or legal mapping. GDPR checklists list duties but are not linked to evidence [34]. This component links each duty to a check and each check to a signed result.

7.3 Research contribution

The main research contribution is a method for checking the privacy evidence of a synthetic data pipeline independently. The method checks each claim with code written apart from the code that made it. It tests every check with a planted fault before trusting a pass. It also gives a missing check its own verdict, so that absence of evidence is not read as a pass.

A second contribution is the finding that calibration budgets must be checked as a total. The earlier release would have passed every per-query test, including the

reconstruction attack. Only the composition check found that seven queries at $\varepsilon = 1.8$ spend 12.6. A third contribution is a documented example of a predictable noise source in a DP release. Its noise had the correct size and passed the size check, but it could be reproduced from a public seed.

7.4 Product contribution

The product contribution is the risk_compliance package with its nine commands. It produces a compliance report and a signed certificate from the evidence of a pipeline. The checks run on a laptop CPU in seconds. This means they can run on every pipeline execution, and not only once before a release. The certificate gives a non-technical reader a one-page verdict. It also gives a technical reader a fingerprint and a command to check it. Inside the project, the component found one real fault and confirmed its fix. It confirmed that the training evidence of Privacy Integration and the dataset of Dataset Generation were intact.

7.5 Limitations and threats to validity

Internal validity. The checks were written by one person and tested with faults that the same person planted. A fault that the author did not think of may pass every check. The fixed seed is an example. The budget audit trusts that the per-step log was written by the training run. It cannot prove that Opacus computed ε correctly. The HMAC key was shared between the producer and the auditor. As a result, either party could create a valid tag. The signature proves integrity against outsiders but not between the two parties.

Construct validity. Some GDPR mappings rest on indirect evidence. For example, Article 17 is linked to k-anonymity. A passing k-anonymity check shows that frames cannot be singled out from metadata. It does not show directly that no individual record is stored. Articles 5(1)(b) and 5(1)(c) are linked to the noise log. The log shows what was queried, not how the data was used later. The reconstruction ratio compares only five values, so it can detect gross under-noising but not small errors. The ratio bounds of 0.3 and 3.0 were chosen by judgment and were not derived from a statistical test.

External validity. The calibration statistics came from a single source, the Waymo Open Dataset. The privacy study of Privacy Integration also trained on all 10,000

frames. It did not use the 7,600 that passed the poisoning gate. The checks were run on one project with one simulator. Other pipelines may produce evidence in other formats.

Scope. Two attacks named in the SRS were not carried out by this component. Privacy Integration ran the membership inference study on its own models. Its private models reached an attack success rate of 50.3% to 51.6%, below the proposal target of 55%. That result was not repeated independently. The re-identification attack was not implemented, so the target of 10% has no measured result. Utility Evaluation also reported a utility gap. The synthetic model reached only 10.46% of the real-data mAP50. The group target was to stay within 15% of the real-data baseline.

7.6 Implications

For teams that build private pipelines, the results suggest that every reported ε needs a second reader. The fault found here was simple to fix but easy to miss. A team should log enough detail for the total budget to be recomputed. It should also treat every random seed of a release mechanism as a secret, or use a secure random source.

For auditors and data protection officers, the results show a clear way to link technical checks to legal duties. A provision is verified only when its check ran and passed. The verdict must also state what it does not cover. A certificate with a fingerprint lets a reader confirm that the verdict was not edited after signing. For regulators, the approach shows how to ask for privacy evidence that can be checked, not only described.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value proposition and target users

The component turns the privacy claims of a data pipeline into evidence that someone else can check. A team that uses it receives three things. These are independent check results, a compliance report that links each result to a GDPR duty and a signed certificate. The certificate gives a reader a one-page verdict and a way to confirm that it was not edited. The report states its limits and the known weaknesses next to the verdict.

The project proposal identified four groups of users. Their needs differ for this component:

- **Autonomous vehicle developers**, mainly data infrastructure leads and machine learning engineers. They need a record they can show to customers and partners before sharing data or models.
- **Data protection officers and regulators**, who need evidence they can read and verify. They also need to know what the evidence does not cover.
- **Academic researchers**, who want to publish synthetic datasets with a privacy record that reviewers can check.
- **Road users**, who benefit when faults such as an overspent budget or a predictable seed are found before release.

8.2 Market and competitive position

The target market is organizations that process personal data to train machine learning models and must show compliance. In the automotive sector, these include autonomous vehicle start-ups, suppliers that build perception systems and simulation service providers. Similar needs exist in health and finance, where privacy-preserving machine learning is also used.

Existing offerings cover parts of the need. Privacy management platforms record processing activities and policies, but they do not run technical checks on pipeline outputs. Model auditing tools such as ML Privacy Meter [11] measure leakage from a trained model. They do not check budget records, data integrity or legal mapping. DP

libraries such as Opacus [32] report their own ε, but nothing checks the report. This component sits between these tools. It reads the outputs of a pipeline, checks each claim and gives a signed verdict mapped to the law.

8.3 Business or sustainability model

The most practical model is an open-source core with paid audit services. The risk_compliance package could be released as an open-source toolkit with a documented evidence format. Open code matters for an auditing tool, because users must be able to see what each check does. Companies could then pay for three services. The first is adapting the checks to their own pipeline and evidence formats. The second is an independent audit run by a separate party that holds the keys. The third is support during an assessment by a data protection authority.

The running costs are low. All checks ran on a laptop CPU in seconds. The main dependencies are free, open-source libraries. The largest cost would be the staff time needed to map each customer's controls to legal duties. This work also needs legal review, which the tool does not replace.

8.4 Scalability intellectual property and compliance

Scalability. The checks scale with the size of the evidence, not with model size. The manifest check reads every file once, so it grows linearly with the dataset. It can run in parallel across towns or folders. The budget audit grows with the number of training steps, which is small. A larger deployment would need a standard evidence format. Pipelines built with other tools could then be checked without new code.

Intellectual property. The component builds on open-source libraries such as pandas and ReportLab. Any release must respect their licenses and keep their notices. The code written for this component belongs to the author and the project, subject to the rules of the university. The GDPR mapping summarizes public law and guidance, so it carries no separate rights.

Compliance. The component itself processes no personal data in its normal use. It must still protect the files it reads. The calibration noise log and the source values must never be published, and the HMAC key must stay secret. The certificate must keep its

statement that it is not a legal certification of GDPR compliance. A product that dropped this statement could mislead its users. A commercial version would also need public-key signatures. With a shared HMAC key, the producer and the auditor can each create a valid tag.

8.5 Contribution to the Sustainable Development Goals

The component contributes most to two Sustainable Development Goals.

- **SDG 16: Peace, Justice and Strong Institutions.** Target 16.6 calls for effective, accountable and transparent institutions. Target 16.10 includes the protection of fundamental freedoms. The component makes privacy claims accountable by linking each one to evidence that others can check.
- **SDG 9: Industry, Innovation and Infrastructure.** Target 9.5 asks for stronger research and technological capability. The component gives developers of synthetic data a method to show that their privacy controls work as claimed.

The link to road safety under SDG 3 (target 3.6) and SDG 11 (target 11.2) is indirect. Trusted access to training data may help perception models improve, but this thesis did not measure any effect on safety. There are also possible negative effects. A PASS verdict could give a false sense of security if users read it without its limits. The fixed seed found in this work is an example of a fault that every check passed. Publishing the declared weaknesses and the scope note with every certificate reduces this risk.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This research built an independent verification component for a privacy-enhanced synthetic data pipeline. The Risk and Compliance component checks the claims of the other three components with code written apart from theirs. It then maps the evidence to the GDPR and issues a signed certificate.

On the final outputs, all twelve signed audit reports were valid, and a changed copy was rejected. Every reported training ε matched its per-step log and stayed below the limit of 10. All 40,000 delivered files matched their manifests. The metadata reached $k = 5$ with no unique rows, and the release leaked no paths or secrets. The final report verified eight GDPR provisions, gave an overall PASS and was signed as certificate R26-RC-F3335EAE8286.

The most important result was a fault that the producing team had missed. The earlier calibration release spent $\varepsilon = 12.6$ while it reported 1.8. The composition check found this, and the same check confirmed the fix. A later code review found a second fault that no check could detect. The calibration noise used a fixed public seed, which was corrected before the recapture. Both results lead to the same conclusion. A privacy claim should be checked by someone other than its author. The limits of the checks themselves must also be stated.

9.2 Achievement of objectives

Table 9.1 compares the results with the objectives in Section 1.5.

Table 9.1: Achievement of objectives

Objective	Outcome	Status
GDPR compliance baseline	Eight provisions mapped to controls, owners and checks	Achieved
Verify signed reports	12 of 12 valid; changed copy rejected	Achieved
Audit every reported ε	9 of 9 private runs agree with their logs; limit kept	Achieved
Check the calibration channel	Overspend found and fix confirmed; ratio 0.94	Achieved; seed fault not detectable by the checks

Objective	Outcome	Status
Check dataset and metadata	40,000 files match; k = 5; no leakage	Achieved
Report and certificate	Overall PASS; signed certificate checked with --verify	Achieved
Pipeline integration	Seven stages with a manifest gate; end-to-end test	Achieved
Membership inference at or below 55% (proposal)	Measured by Privacy Integration (50.3% to 51.6%); not repeated independently	Not done by this component
Re-identification at or below 10% (proposal)	Attack not implemented	Not done

9.3 Individual research and product contributions

The individual research contributions of this thesis are as follows:

- A method for checking the privacy evidence of a pipeline independently. Each check is written apart from the code it tests and is tested with a planted fault.
- Evidence that calibration budgets must be checked as a total. Seven correctly noised queries at $\varepsilon = 1.8$ spent 12.6, and only the composition check found this.
- A documented example of a DP release whose noise had the right size but came from a fixed public seed.
- A verdict rule that keeps a missing check from being read as a pass.

The individual product contributions are as follows:

- The risk_compliance package with ten modules and nine commands.
- The GDPR mapping with eight provisions and two declared weaknesses.
- The compliance report and the signed PDF certificate with a QR fingerprint and a verify command.
- Seven pipeline stages, including the manifest gate, and 79 unit tests.

9.4 Future directions

The future work is listed in order of priority.

1. **Seed check.** Add a check that fails when a release mechanism uses a fixed or public seed. A fault like the one in this work would then be found automatically.
2. **Independent membership inference.** Repeat the membership inference study with separate code, using the reserved attack dependencies of the package. Compare the result with that of Privacy Integration.
3. **Public-key signatures.** Replace HMAC with public-key signatures, so that only the auditor can sign and anyone can verify.
4. **Stronger anonymity measures.** Add l-diversity [29] and a margin above the k threshold, and examine what the images themselves reveal.
5. **Wider mapping.** Extend the mapping to the NIST Privacy Framework [35]. Confirm each summary against the text of the regulation with a legal reviewer.

REFERENCES

[1]A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? The KITTI vision benchmark suite," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2012, pp. 3354-3361.

[2]P. Sun *et al.*, "Scalability in perception for autonomous driving: Waymo Open Dataset," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 2446-2454.

[3] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation)," *Off. J. Eur. Union*, vol. L 119, pp. 1-88, May 2016.

[4]A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Proc. 1st Annu. Conf. Robot Learn. (CoRL)*, ser. Proc. Mach. Learn. Res., vol. 78, 2017, pp. 1-16.

[5]R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2017, pp. 3-18.

[6]T. Stadler, B. Oprisanu, and C. Troncoso, "Synthetic data: Anonymisation groundhog day," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 1451-1468.

[7]F. Tramèr, A. Terzis, T. Steinke, S. Song, M. Jagielski, and N. Carlini, "Debugging differential privacy: A case study for privacy auditing," 2022, *arXiv:2202.12219*.

[8]Z. Ding, Y. Wang, G. Wang, D. Zhang, and D. Kifer, "Detecting violations of differential privacy," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2018, pp. 475-489.

[9]M. Jagielski, J. Ullman, and A. Oprea, "Auditing differentially private machine learning: How private is private SGD?" in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 22205-22216.

[10]M. Nasr, S. Song, A. Thakurta, N. Papernot, and N. Carlini, "Adversary instantiation: Lower bounds for differentially private machine learning," in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2021, pp. 866-882.

[11] S. K. Murakonda and R. Shokri, "ML Privacy Meter: Aiding regulatory compliance by quantifying the privacy risks of machine learning," 2020, *arXiv:2007.09339*.

[12]C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating noise to sensitivity in private data analysis," in *Proc. 3rd Theory Cryptogr. Conf. (TCC)*, ser. Lecture Notes Comput. Sci., vol. 3876. Berlin, Germany: Springer, 2006, pp. 265-284.

[13]C. Dwork, "Differential privacy," in *Proc. 33rd Int. Colloq. Automata, Lang. Program. (ICALP)*, ser. Lecture Notes Comput. Sci., vol. 4052. Berlin, Germany: Springer, 2006, pp. 1-12.

[14]C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," *Found. Trends Theor. Comput. Sci.*, vol. 9, no. 3-4, pp. 211-407, 2014.

[15]M. Abadi *et al.*, "Deep learning with differential privacy," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2016, pp. 308-318.

[16]I. Mironov, "Rényi differential privacy," in *Proc. IEEE 30th Comput. Secur. Found. Symp. (CSF)*, 2017, pp. 263-275.

[17] *Secure Hash Standard (SHS)*, FIPS PUB 180-4, Nat. Inst. Standards Technol., Gaithersburg, MD, USA, 2015.

[18]H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," Internet Eng. Task Force, RFC 2104, Feb. 1997.

[19]P. C. Kocher, "Timing attacks on implementations of Diffie-Hellman, RSA, DSS, and other systems," in *Advances in Cryptology (CRYPTO)*, ser. Lecture Notes Comput. Sci., vol. 1109. Berlin, Germany: Springer, 1996, pp. 104-113.

[20]L. Sweeney, "k-anonymity: A model for protecting privacy," *Int. J. Uncertainty, Fuzziness Knowl.-Based Syst.*, vol. 10, no. 5, pp. 557-570, 2002.

[21]Article 29 Data Protection Working Party, "Opinion 05/2014 on anonymisation techniques," WP216, Brussels, Belgium, Apr. 2014.

[22] A. Cohen and K. Nissim, "Towards formalizing the GDPR's notion of singling out," *Proc. Nat. Acad. Sci. USA*, vol. 117, no. 15, pp. 8344-8352, 2020.

[23]I. Mironov, "On significance of the least significant bits for differential privacy," in *Proc. ACM Conf. Comput. Commun. Secur. (CCS)*, 2012, pp. 650-661.

[24] I. Dinur and K. Nissim, "Revealing information while preserving privacy," in *Proc. 22nd ACM SIGMOD-SIGACT-SIGART Symp. Princ. Database Syst. (PODS)*, 2003, pp. 202-210.

[25]B. Schneier and J. Kelsey, "Secure audit logs to support computer forensics," *ACM Trans. Inf. Syst. Secur.*, vol. 2, no. 2, pp. 159-176, 1999.

[26]S. A. Crosby and D. S. Wallach, "Efficient data structures for tamper-evident logging," in *Proc. 18th USENIX Secur. Symp.*, 2009, pp. 317-334.

[27]A. Narayanan and V. Shmatikov, "Robust de-anonymization of large sparse datasets," in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2008, pp. 111-125.

[28]L. Rocher, J. M. Hendrickx, and Y.-A. de Montjoye, "Estimating the success of re-identifications in incomplete datasets using generative models," *Nature Commun.*, vol. 10, Art. no. 3069, 2019.

[29]A. Machanavajjhala, D. Kifer, J. Gehrke, and M. Venkitasubramaniam, "l-diversity: Privacy beyond k-anonymity," *ACM Trans. Knowl. Discov. Data*, vol. 1, no. 1, Art. no. 3, 2007.

[30]T. Gu, B. Dolan-Gavitt, and S. Garg, "BadNets: Identifying vulnerabilities in the machine learning model supply chain," 2017, *arXiv:1708.06733*.

[31]M. Meli, M. R. McNiece, and B. Reaves, "How bad can it git? Characterizing secret leakage in public GitHub repositories," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2019.

[32]A. Yousefpour *et al.*, "Opacus: User-friendly differential privacy library in PyTorch," 2021, *arXiv:2109.12298*.

[33] *Security Techniques: Extension to ISO/IEC 27001 and ISO/IEC 27002 for Privacy Information Management: Requirements and Guidelines*, ISO/IEC 27701:2019, Int. Org. Standardization, Geneva, Switzerland, 2019.

[34] European Data Protection Board, "Guidelines 4/2019 on Article 25: Data protection by design and by default, Version 2.0," Brussels, Belgium, Oct. 2020.

[35] Nat. Inst. Standards Technol., "NIST Privacy Framework: A tool for improving privacy through enterprise risk management, Version 1.0," U.S. Dept. Commerce, Gaithersburg, MD, USA, 2020.

GLOSSARY

Differential privacy. A property of a randomized computation that limits how much any single record can change its output.

Privacy budget (ε). The number that sets the strength of differential privacy. A smaller ε means stronger protection and more noise.

Sensitivity (Δf). The largest change that one record can cause in the answer to a query.

Laplace scale (b). The scale of the Laplace noise, equal to $\Delta f / \varepsilon$. It is also the mean absolute value of the noise.

Sequential composition. The rule that the budgets of several private queries on the same records add up.

Calibration ratio. The mean observed reconstruction error divided by the mean expected error. A value near 1.0 means the noise had the required size.

HMAC. A tag computed from a message and a secret key. Only a key holder can produce a valid tag.

Digest. A fixed-length fingerprint of a file computed with a hash function such as SHA-256.

Tamper test. A test that changes a copy of a signed file and confirms that verification rejects it.

Manifest. A list of the SHA-256 digests of every file in a delivery.

Quasi-identifier. An attribute that is not unique alone but can identify a record in combination with others.

k-anonymity. The property that every record shares its quasi-identifiers with at least $k - 1$ other records.

Singling out. Isolating one record in a dataset, which anonymization must prevent.

Declared weakness. A known limit of the pipeline that is written into every report.

APPENDICES

Appendix A Extended results

This appendix gives the evidence behind Chapter 6 in more detail. Table A.1 lists the manifest check for each town.

Table A.1: Manifest verification per town

Town	Manifest entries	Verified	Mismatched	Missing
Town01	8,000	8,000	0	0
Town02	8,000	8,000	0	0
Town03	8,000	8,000	0	0
Town04	8,000	8,000	0	0
Town05	8,000	8,000	0	0

Table A.2 lists the composition check of the corrected calibration release. Each per-query ε was recovered from the noise log as Δf divided by the Laplace scale.

Table A.2: Composition check of the corrected calibration release

Query	Δf	Laplace scale b	ε spent
Vehicle density per km	1.0	3.888889	0.257143
Pedestrian density per km	1.0	3.888889	0.257143
Average vehicle speed (km/h)	2.0	7.777778	0.257143
Average pedestrian speed (km/h)	1.0	3.888889	0.257143
Intersections per km	1.0	3.888889	0.257143
Weather distribution	1.0	3.888889	0.257143
Time-of-day distribution	1.0	3.888889	0.257143
Total under sequential composition			1.8

In the earlier release, each query had a Laplace scale of Δf / 1.8. This is 0.555556 for Δf = 1. The recovered ε was 1.8 per query and 12.6 in total. The noise log of that release stored the scale in a field named epsilon_used. Section 5.2.1 explains how the check handled this.

Table A.3 lists the result of the poisoning gate for each town. The values come from the poisoning audit of Utility Evaluation.

Table A.3: Poisoning gate result per town

Town	Frames	Accepted	Excluded	Cyclist share of boxes
Town01	2,000	1,400	600	0.030
Town02	2,000	1,500	500	0.103
Town03	2,000	1,500	500	0.120
Town04	2,000	1,500	500	0.123
Town05	2,000	1,700	300	0.119

The certificate record holds the following values. The SHA-256 of compliance_report.json is f3335eae82869fe92135d6418bf7c12bd8ff4352d9f3d5b6c81dc0c11fb095ff. The HMAC-SHA256 signature of this value is a44a8de8937d87fdab9847d98551008c561a14fdcb5099aeaa47139c451f1ef1. The SHA-256 of the PDF is 734b0afa67fbf21a80fd9438d1d45073a75186dd922b3c6c86d0d6b5be3cdbd5. The certificate was issued at 01:32:24 UTC on 20 October 2026.

Appendix B Academic integrity and AI use disclosure

Generative AI was used while preparing this thesis, in line with the module policy on AI use.

Tool. Claude, a large language model by Anthropic.

Use. The tool drew the diagrams and charts from the author's results. It was also used for a final verification of the thesis before submission.

Verification. The author checked the numbers, citations and claims against the experiment outputs and the cited sources.

Responsibility. The research design, the experiments, the results and the conclusions are the responsibility of the author.